# Nimbus — How It Works

**Version:** 1.2 **Author:** Sujal Kumar Singh **Last updated:** 2026-08-18

This is the plain-English version. Read this to understand the system.

| Doc | For |
|---|---|
| `OVERVIEW.md` (this one) | Understanding what we are building and why |
| `ARCHITECTURE.md` | Every part of the system drawn as a diagram |
| `HLD.md` | Exact table columns, endpoints, and steps. The build reference. |

---

## 1. The whole thing in one line

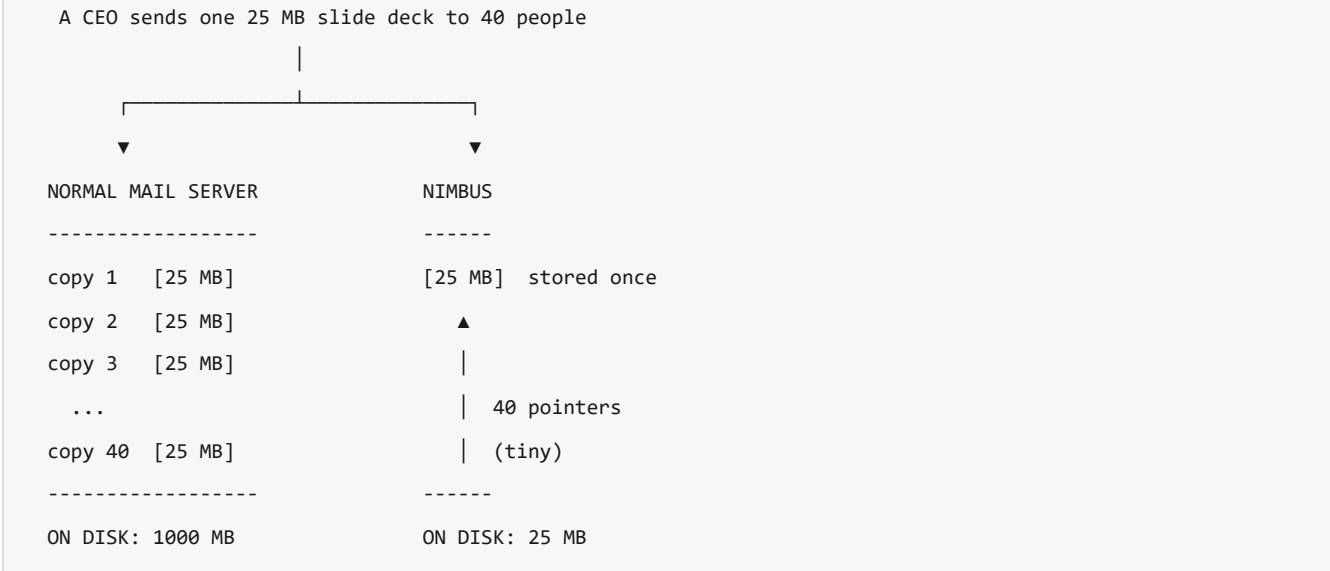
> A mail server that stores every attachment once, no matter how many people receive it.

---

## 2. The problem

Business email costs about **$2 per mailbox per month**. At that price, the biggest cost is storage. So storage is where you win or lose money.

Here is what goes wrong with a normal mail server:




Same email. Same 40 people. **40x less disk.**

Now imagine 10 million mailboxes instead of 40. That gap decides whether the business makes money or not.

---

## 3. The trick

We never ask "*what is this file called?*". We ask "*what is this file made of?*".

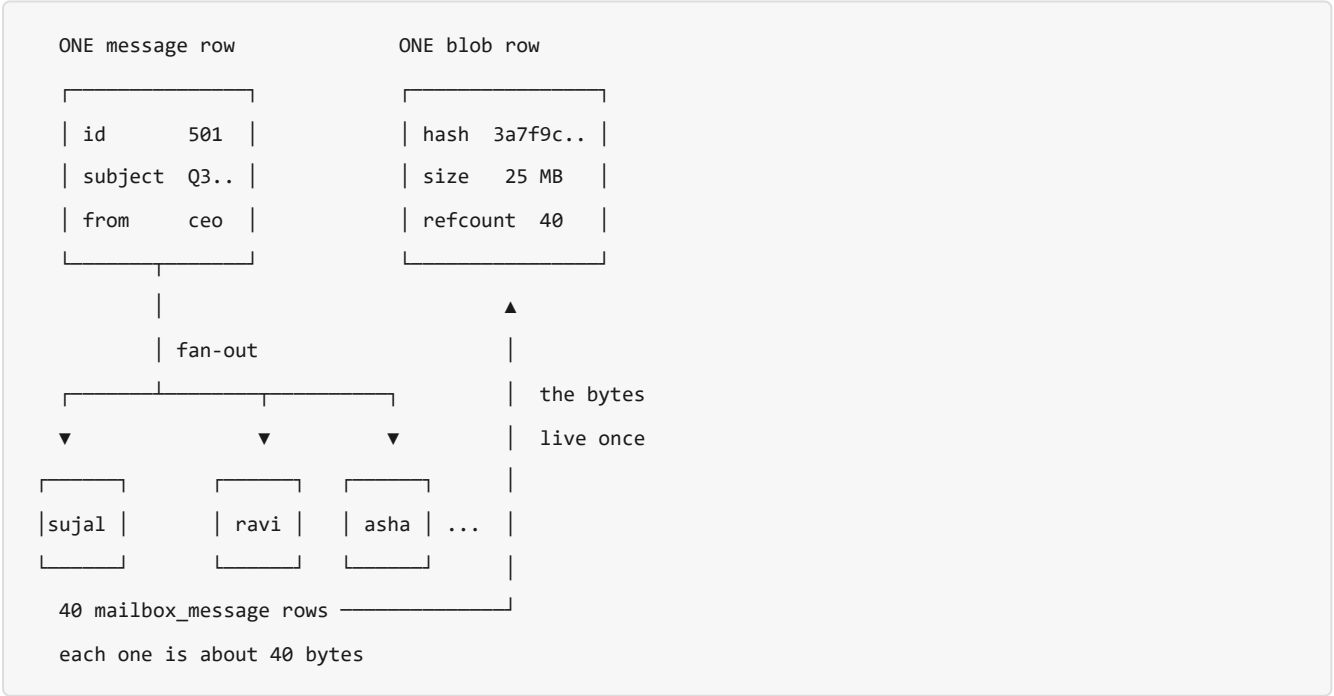
```
any bytes  ──►  SHA-256  ──►  3a7f9c2e...d41   (64 characters)

same bytes       ──►  same fingerprint  ──►  we have it, store nothing
one bit differs  ──►  totally different ──►  new file, store it
```

That fingerprint is the file's name in our storage. This is called **content-addressed storage**.

Three things fall out of it for free:

- **No duplicates.** Same fingerprint means we already have those bytes.
- **Writing twice is safe.** Two people uploading the same file at the same moment both write to the same place. No conflict, no lock needed.
- **Damage is detectable.** If the bytes ever change, the fingerprint stops matching.

### How the data actually sits




- **1 message row** — the email itself, stored once
- **1 blob row** — the attachment, stored once
- **40 tiny rows** — one per person, so each person sees it in their own inbox

Everyone gets their own copy in their inbox. Nobody gets their own copy on disk.

---

## 4. Goals — what "done" means

| # | Goal | Done when |
|---|---|---|
| 1 | Accept real email | A real mail server can send us mail and we keep it |
| 2 | Store each file once | Same attachment to 40 people = 1 copy on disk |
| 3 | Deliver to many, copy nothing | 40 recipients = 40 small rows, 0 extra bytes |
| 4 | Search properly | `from:boss has:attachment invoice` answers in under 100 ms |
| 5 | Show what we saved | Dashboard: "they think 1 GB, we stored 25 MB" |
| 6 | Free space safely | Deleting the last copy frees the disk, and never too early |
| 7 | Let a reseller sign customers up | One API call creates a domain and its mailboxes |

---

## 5. What we are NOT building

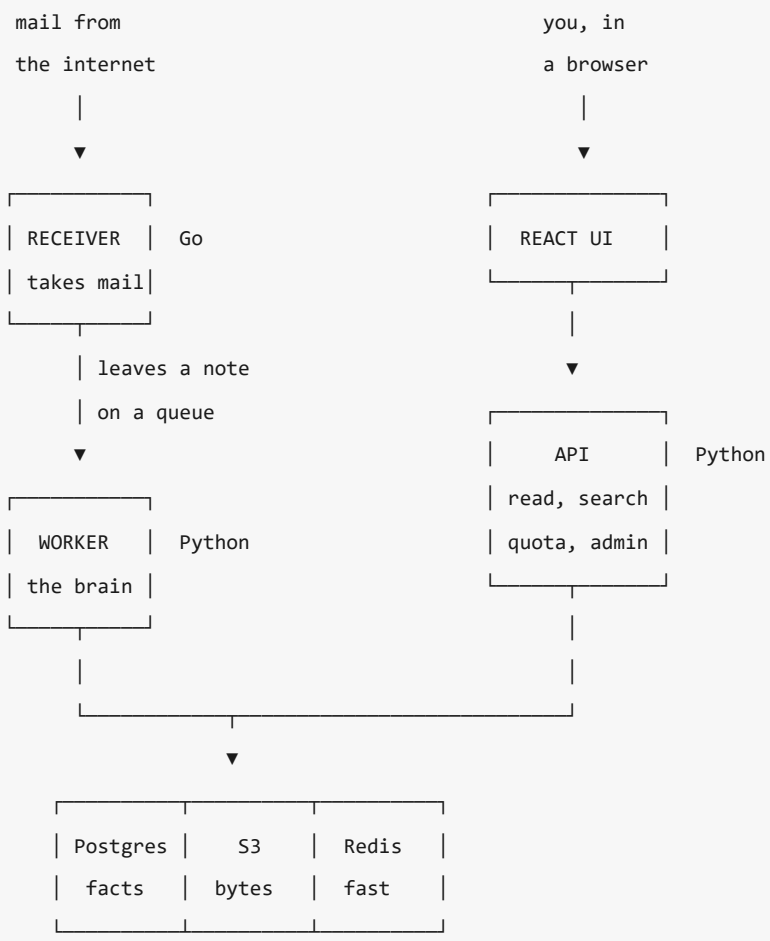
This matters as much as the goals. It stops the project growing forever.

- **We do not send email.** No outbound. Nimbus only receives.
- No Send Later, no Unsend, no read receipts — all of those need sending.
- No calendar, contacts, signatures, templates.
- No IMAP or POP3. Browser only.
- No spam filtering beyond basic sender checks.
- No mobile apps.

**Why receive-only?** Because dedup only helps when you *store*. When you send a 25 MB file to 40 people, SMTP makes you transmit all 1000 MB anyway. There is no trick for that. Storage is where the saving is real, so that is where we work.

---

## 6. The parts




```
plus one small background job:
   CLEANUP (Python)  deletes files nobody points at any more

There is no snooze worker. A snoozed message simply has a
"hide until" time on it, and the inbox stops hiding it when
that time passes. Nothing has to wake up and do anything.
```

| Part | Language | Its one job | Why that language |
|---|---|---|---|
| Receiver | Go | Take mail off the wire, put it in S3 | Many slow connections at once, flat memory. Capped at 100 so a flood cannot exhaust the box. |
| Worker | Python | Split, fingerprint, dedup, deliver, index | The thinking part. Nothing here is urgent. |
| API | Python | Everything the browser asks for | Most of the code. Speed of writing matters most. |
| Cleanup | Python | Delete unused files | Runs in the background. Slow is fine. |

### Where things are kept

| Store | Holds | Why there |
|---|---|---|
| **Postgres** | Who owns what, who got which email, all counts | Needs to be exactly right. Source of truth. |
| **S3** | The actual bytes of files and emails | Files must never go in a database. Cheap, endless. |
| **Redis** | Which addresses exist, so the receiver can answer `RCPT TO` in under a millisecond | It is a **cache** — rebuilt from Postgres at startup, so nothing lives here that we could not rebuild |
| **Kafka** | The queue of "new mail arrived" notes | If a traffic spike hits, mail waits instead of being lost |

---

## 7. What happens when an email arrives

```
1  Another mail server connects to us
             |
2  It says "I have mail for sujal@acme.com"
   We check right now: does that address exist?
      no  ──►  we reply "550 No such user"   <-- our ONLY chance to say no
      yes ──►  carry on
             |
3  The email arrives. We write it straight into S3
   as it flows in. We never hold the whole thing in memory.
             |
4  We leave a note on the queue: "new mail, here is where
   it is". Then we hang up. The sender is done.
             |
5  A worker picks up the note and does the real work:
      - split the email into text and attachments
      - fingerprint each attachment
      - do we already have it?
            yes  ──►  store nothing at all
            no   ──►  upload it
      - work out who actually gets it (see section 8)
      - write one small row per person
      - add it to the search index
```

**Why step 2 matters so much.** Step 2 is the last moment we can refuse a message. After that the sender has hung up and gone. By step 5 there is nobody left to talk to. So every check that could reject mail has to happen at step 2, not later.

**Why step 3 matters.** A 25 MB email must not use 25 MB of memory. We copy it through to S3 in small pieces as it arrives. 1 email or 1000 at once, memory stays flat.

**Why step 4 matters.** Once the note is on the queue, the mail is safe. If the worker crashes, the note is still there and gets picked up again. Nothing is lost.

**One catch with step 5.** The queue can hand us the same note twice — that is normal and expected. If we just processed it again, the same email would appear in someone's inbox twice and our counts would go wrong. So the worker writes down "I have done this one" first, in the same database transaction as everything else. A repeat instantly collides with that record and stops.

---

## 8. Who actually gets the email

An address is not always a person. We check in this order and stop at the first match.

```
mail for sales@acme.com
          |
1  Is it a nickname for someone?     ──► send to that person
          | no
2  Is it a shared inbox?             ──► send to every member
          | no
3  Does it forward somewhere?        ──► forward it on
          | no
4  Does the domain catch everything? ──► drop it in that inbox
          | no
5  Nothing matched                   ──► log it and drop it
```

Step 5 is not a bounce, and it cannot be. We already said yes to this address back in section 7 step 2. The connection is gone. Sending a real bounce email would mean sending mail, which section 5 says we do not do. In practice step 5 only happens if someone deleted the mailbox in the few seconds in between.

---

## 9. Opening an email with a big attachment

The file might be 25 MB. We do not load 25 MB to hand it over.

```
blob 3a7f9c..  is stored as pieces:

  ┌────────┐ ┌────────┐ ┌────────┐ ┌────────┐
  | 4 MB   | | 4 MB   | | 4 MB   | | 4 MB   | ...
  └────────┘ └────────┘ └────────┘ └────────┘
      1          2          3          4

We read piece 1, send it, forget it.
Then piece 2. Then piece 3.

Memory used: one piece at a time. Never the whole file.
```

This also means a download can resume if the connection drops.

---

## 10. Deleting things without losing data

This is the hardest part of the whole project.

Every stored file counts how many people still point at it. That count is the **refcount**.

```
refcount = how many inboxes still hold this file

 40 ──► someone deletes ──► 39 ──► ... ──► 0
                                           |
                                    wait 24 hours
                                           |
                                    now delete the bytes
```

**Why wait 24 hours instead of deleting at zero?**

```
Mailbox A:  deletes its copy         refcount goes 1 ──► 0
Mailbox B:  is uploading the SAME    (at that exact moment)
            file right now

Delete immediately:  we wipe the bytes B is about to point at.
                     B now has a broken attachment.

Wait 24 hours:       B's upload pushes the count back to 1
                     long before the sweep runs. Nothing breaks.
```

The delay costs us a little disk space for a day. It saves us from needing a global lock across the entire system, which would be far more expensive and far more fragile.

## 11. What the user sees vs what we store

| | Meaning | Example |
|---|---|---|
| **Logical** | What the user thinks they used | 1 GB |
| **Physical** | What we actually put on disk | 25 MB |

**We bill logical.** Three reasons:

- The user's mental model is "my mailbox holds 1 GB". Billing anything else is confusing.
- The saving is our profit margin, not a discount we give away.
- Physical is what we track to prove the engine works.

The dashboard shows both side by side. That contrast is the demo.

---

## 12. The hard problems, in plain words

| Problem | What we do |
|---|---|
| Two people get the same file at the same second | Fingerprints make writing twice harmless. A short Redis lock stops us uploading it twice for nothing. |
| When is it safe to delete? | Count the pointers. Wait 24 hours at zero. |
| Encryption breaks dedup | The same file encrypted with two different keys looks like two different files. For now we use one key for storage, so dedup still works. Noted trade-off. |
| Dedup can leak information across customers | If an upload is suspiciously instant, you learn someone else already has that file. Real published attack on Dropbox. **We fix it by only deduplicating within one customer, never across.** |
| Receiving 25 MB without using 25 MB of RAM | Stream it through in pieces. Never hold the whole thing. |
| 4 MB pieces — is that the right size? | For most email, attachments are smaller than 4 MB anyway, so this rarely matters. It matters for big files. Smarter splitting is an upgrade later if we need it. |

---

## 13. How we know it worked

All six have been measured.

| What we measure | Target | What we got |
|---|---|---|
| Disk saved on realistic mail | more than 60% | **68.8%** on 10,000 emails |
| Mail accepted per minute | more than 500, kept up | **500, kept up** — never more than 6 emails behind |
| Memory while receiving 25 MB files | flat — does not grow with file size | 212 MB of mail through 84 MB of memory |
| Search on a 100,000-message mailbox | under 100 ms for 95% of searches | **5 ms** for an ordinary word, **73–76 ms** for a search matching nearly the whole mailbox |
| Snooze accuracy | **exact** — nothing fires, so nothing can be late | exact |
| Timers we can hold at once | 1 million, no slowdown | nothing to slow down — they are just rows |

**Important honesty note on the first row.** That number is decided by the test emails we generate, not by the code. Random files dedup at 0%. All-identical files dedup at 100%. Neither proves anything. So we write down exactly what the test set contains and publish it next to the number — and the whole test set is reproducible from a single number, the "seed". Change the seed and the same recipe gives anything from **65.7% to 78.3%**. That swing is not noise to hide; it is the honest width of the claim.

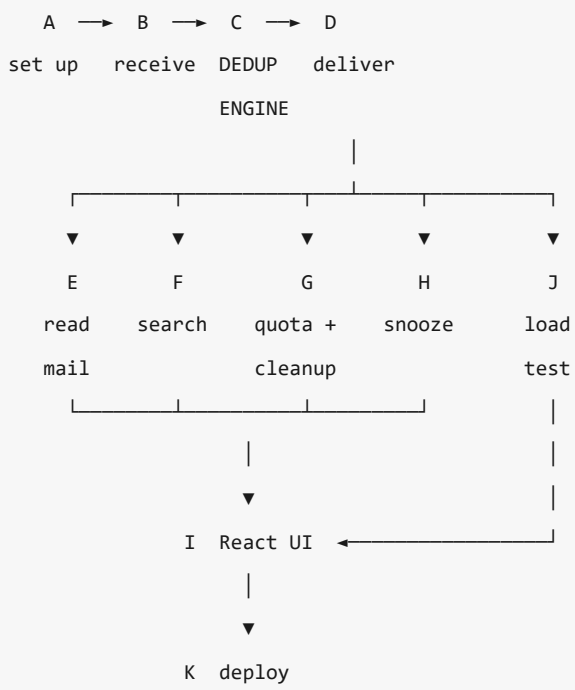
**And a second honesty note, on what "saved" counts.** 68.8% is the saving on *attachments*. We also keep the original raw email for 7 days, and that copy is not deduplicated at all — forty emails carrying one slide deck keep forty full copies of it. Measured, that raw pile was **4.4 times bigger** than the deduplicated store. So the saving on the disk you actually pay for is **68.4% today, and 94.2% once the 7-day copies expire**. The bigger number is real, it just arrives a week late.

**We also found the maximum.** 500 a minute is what we *asked* for, and the system kept up without straining. So we asked for 6,000 a minute to see where it breaks. It stored **1,417 a minute** — nearly three times the target — and the part that ran out of road was the **worker** (the brain), not the receiver (the part taking mail off the wire). The receiver was still accepting twice what the worker could store.

The important half of that result is what did *not* happen. At three times the target the queue built up to 5,777 waiting emails — and then drained to zero, with every one of the eleven correctness checks still passing and not a single email lost. Overloading it made it slow, not wrong. That is exactly the job the queue exists to do.

---

## 14. Build order




| Block | What | Notes |
|---|---|---|
| A | Docker, database, login, sign-up API | **Done.** Everything waits on this |
| B | The SMTP receiver | **Done.** 212 MB of mail through 84 MB of memory |
| C | **The dedup engine** | **Done.** The point of the whole project |
| D | Work out who gets what, write the rows | **Done.** |
| E | Read your inbox, download attachments | **Done.** |
| F | Search | **Done.** `from:boss has:attachment invoice` works |
| G | Quota and cleanup | **Done.** The sweep that actually gives the disk back |
| H | Snooze | **Done.** No worker — the mail just comes back when the time passes |
| I | The React screens and the savings dashboard | |
| J | Load test with 100,000 emails | Takes ~3.5 hours of real time. Start it early. |
| K | Deploy to AWS | |

**Two things take real-world time and cannot be rushed:**

1. The load test itself runs for about 3.5 hours. Start it as soon as D works and build other things while it runs.
2. AWS blocks port 25 on new accounts and takes several days to unblock it. **File that request before writing any code.** It is free and it is slow.

## 15. Words you need

| Word | What it means |
|---|---|
| **SMTP** | The language mail servers use to hand email to each other |
| **MIME** | The format inside an email that lets it carry text and files together |
| **MX record** | A DNS entry saying "mail for this domain goes to that server" |
| **Blob** | One whole attachment |
| **Chunk** | A 4 MB piece of a blob |
| **SHA-256** | Turns any bytes into a fixed 64-character fingerprint |
| **Content-addressed** | Naming a file by its fingerprint instead of its filename |
| **Dedup** | Storing identical content only once |
| **Refcount** | How many things still point at a file |
| **Fan-out** | One email creating rows in many inboxes |
| **Idempotent** | Doing it twice has the same result as doing it once |
| **Multi-tenant** | Many separate customers sharing one system safely |
| **Stream** | Move data through in small pieces instead of loading it all |
| **Grace period** | A deliberate delay before deleting, to avoid a race |
| **p95** | 95 out of 100 requests were faster than this |

## 16. Where to go next

| You want | Read |
|---|---|
| Exact database columns | `HLD.md` §8 |
| Exact API endpoints and login | `HLD.md` §10 |
| Step-by-step flows with real detail | `HLD.md` §9 |
| Decisions still open | `HLD.md` §16 |
| How we work on this project | `../CLAUDE.md` |